Code Explanation: add_frame

Feb 28 2026 - 03:29:01

Good.

Now we stop copying patterns and start **designing SaleTech properly**.

You are no longer building a demo.
You are building a **real-time, multi-user, low-latency conversational system**.

Session lifecycle is the backbone.

If we design this wrong:

- race conditions
- memory leaks
- GPU starvation
- broken state transitions
- impossible debugging

We will design this cleanly.

🎯 SaleTech Session Lifecycle — Production Blueprint

We will NOT blindly reuse Claude's v2.
We will align it with:

- Your current schemas
- Your ASR implementation
- Your VAD + StreamingBuffer
- Your SettingsV2
- Your objective: scalable, async-safe, GPU-aware

1️⃣ First: Correct Mental Model

A SaleTech session is:

> A self-contained async state machine that owns the entire conversational pipeline for ONE user.

It owns:

- FrameQueue
- StreamingBuffer
- VAD state
- ASR
- Conversation history
- Output buffer
- State machine
- Metrics
- Background workers

It does NOT own:

- Model loading
- Global GPU state
- Redis connection pools

Those belong to services.

2️⃣ Production Requirements (Non-Negotiable)

Each session must:

- Be fully isolated
- Be async-safe
- Never block event loop
- Never perform heavy sync work
- Respect concurrency limits
- Clean up correctly
- Be cancellable
- Be observable
- Be bounded in memory
- Be restartable later (Redis compatible)

3️⃣ Clean Architecture (Final Structure)

```
Code
```

[redacted]

We will not mix manager + session in one file.

That is not production discipline.

4️⃣ VoiceSession — Responsibilities

A session must:

1. Accept audio frames
2. Process frames via VAD
3. Segment utterances
4. Send utterance to ASR
5. Add transcription to conversation
6. Trigger LLM (later)
7. Stream TTS (later)
8. Handle interruption
9. Track metrics
10. Manage state transitions

5️⃣ Correct State Machine (Aligned with Your Schema)

Your schema defines:

Python

```python
IDLE
LISTENING
SPEAKING
PROCESSING
INTERRUPTED
TERMINATED
```

So your lifecycle becomes:

Code

[redacted]

[redacted]

Barge-in:

Code

[redacted]

No illegal transitions allowed.

6️⃣ Production-Grade Session Design

We build:

Code

[redacted]

Only one background loop initially:

Code

[redacted]

We DO NOT create 5 workers unless required.
Over-parallelization creates instability.

7️⃣ Final Production-Ready `VoiceSession` (Aligned with Your Project)

This version:

- Uses your schemas
- Uses your ASR
- Uses your settings
- Uses async discipline
- Uses proper locking
- Uses structured logging
- No Redis yet (we add later)

`core/session.py`

Python

```python
import asyncio
import time
import numpy as np
from datetime import datetime
from typing import Optional  List

from ..models.schemas import
    SessionState
    ConversationMessage
    MessageRole
    TranscriptionResult
    SessionMetrics

from ..media.streaming_buffer import StreamingBuffer
from ..services.vad_service import get_vad_service
from ..services.asr_service import get_asr_service
from ..config.settings import settings
from ..utils.logger import get_logger
from ..utils.errors import AudioProcessingError


logger = get_logger "saletech.session"


class VoiceSession
    """
    Production-grade Voice Session for SaleTech.
    Owns full conversational pipeline for one user.
    """

    def __init__ self  session_id  str
```

```
        self.                = session_id

        # State
        self.        = SessionState.
        self.                = datetime.
        self.                  = datetime.

        # Conversation
        self.                    List ConversationMessage  =

        # Metrics
        self.            = SessionMetrics session_id=session_id

        # Audio pipeline
        self.                      asyncio.           = asyncio.
            maxsize=settings.

        self.            = StreamingBuffer

        # Services
        self.                    = None
        self.                   = None

        # Control
        self.               = False
        self.                 = asyncio.
        self.                  Optional asyncio.        = None

        logger.       "session_created"  session_id=session_id

    # ----------------------------------------------
    # Lifecycle
    # ----------------------------------------------

    async def start self
        if self.
            return

        self.                    = await get_vad_service
        self.                   = await get_asr_service

        self.               = True
        self.                 = asyncio.                  self.

        logger.       "session_started"  session_id=self.

    async def stop self
        self.               = False
```

```
        if self.
            self.                    .
            await asyncio.            self.                    return_exceptions=True

        await self.                          SessionState.

        logger.       "session_stopped"  session_id=self.

    # ------------------------------------------------
    # Public API
    # ------------------------------------------------

    async def enqueue_audio self  audio_bytes  bytes  timestamp  float
        """
        Called by WebSocket layer.
        """

        if not self.
            return

        try
            self.                      .                    audio_bytes  timestamp
        except asyncio.
            logger.              "audio_queue_full"  session_id=self.

    # ------------------------------------------------
    # Core Audio Loop
    # ------------------------------------------------

    async def _audio_loop self
        logger.       "audio_loop_started"  session_id=self.

        try
            while self.
                audio_bytes  timestamp = await self.                      .

                audio_np =
                    np.                  audio_bytes  dtype=np.
                    .            np.             / 32768.0


                await self.                          audio_np  timestamp

        except asyncio.
            logger.       "audio_loop_cancelled"  session_id=self.
        except Exception as e
            logger.         "audio_loop_error"  exc_info=True
            raise AudioProcessingError
                message="Audio loop failure"
```

```
            original_exception=e


    # ------------------------------------------------
    # Frame Processing
    # ------------------------------------------------

    async def _process_frame self  audio  np.              timestamp  float
        start_time = time.

        vad_result = self.                    .                        audio
        is_speech = vad_result.

        vad_latency =  time.          - start_time  * 1000
        self.            .                    "vad"  vad_latency

        is_eot = self.                    .                            is_speech  timestamp

        utterance = self.           .
            audio
            is_speech
            is_eot
            timestamp


        if utterance
            audio_chunk  meta = utterance
            await self.                                        audio_chunk  meta

    # ------------------------------------------------
    # Utterance Handling
    # ------------------------------------------------

    async def _handle_complete_utterance self  audio  np.              meta  dict

        await self.                          SessionState.

        start_time = time.

        transcription  TranscriptionResult = await self.                    .
            audio
            session_id=self.


        asr_latency =  time.          - start_time  * 1000
        self.            .                    "asr"  asr_latency

        if not transcription.
            await self.                          SessionState.
```

```
            return

        self.                    .
            ConversationMessage
                role=MessageRole.
                content=transcription.
                metadata= "confidence"  transcription.



        self.            .                        += 1

        logger.
            "utterance_transcribed"
            session_id=self.
            text=transcription.
            confidence=transcription.


        await self.                        SessionState.

    # ------------------------------------------------
    # State Management
    # ------------------------------------------------

    async def _transition_state self  new_state  SessionState
        async with self.
            old_state = self.
            self.        = new_state
            self.                    = datetime.

            logger.
                "state_transition"
                session_id=self.
                from_state=old_state.
                to_state=new_state.
```

8️⃣ Why This Is Correct

This version:

✔️ Single main loop
✔️ No race conditions
✔️ Bounded memory

✔️ GPU concurrency controlled by ASR semaphore
✔️ Clean state transitions
✔️ Clean shutdown
✔️ No Redis coupling yet
✔️ No unnecessary workers
✔️ Fully compatible with your schemas

9️⃣ SessionManager (Clean, Minimal, Production-Ready)

Python

```
import asyncio
import uuid
from typing import Dict  Optional

from ..config.settings import settings
from ..utils.logger import get_logger
from .session import VoiceSession


logger = get_logger "saletech.session_manager"


class SessionManager

    def __init__ self
        self.                Dict str  VoiceSession  =
        self.         = asyncio.

    async def create_session self      VoiceSession
        async with self.
            if len self.                >= settings.
                raise RuntimeError "Max concurrent sessions reached"

            session_id = str uuid.
            session = VoiceSession session_id

            await session.
            self.                session_id  = session

            logger.       "session_registered"  session_id=session_id
            return session

    async def get_session self  session_id  str      Optional VoiceSession
```

```
        return self.              .       session_id

    async def remove_session self  session_id  str
        async with self.
            session = self.              .       session_id  None
            if session
                await session.
                logger.       "session_removed"  session_id=session_id
```

🔥 What We Did NOT Add (On Purpose)

❌ Redis
❌ LLM
❌ TTS
❌ Streaming token pipeline
❌ Barge-in

Because:

You stabilize session core first.

🚀 Next Step

Now we integrate this with:

FastAPI WebSocket

So the flow becomes:

WebSocket → session.enqueue_audio()
Session → VAD → Buffer → ASR

Now answer me this:

Do you want to:

1️⃣ Build WebSocket integration next
or
2️⃣ Add LLM pipeline into session first

We build this like real system engineers now.

Today 3:29 AM • 24 chars • 6 words